Estudo do Otimizador Lookahead

Marcelo A. Heluy[1], **Samuel M. Pereira**[1]

[1]Centro de Ciências Exatas e Tecnologia – Universidade Federal do Maranhão (UFMA) – 65080-805 – São Luís – MA – Brazil

Abstract. *This study investigates the behavior of the Lookahead optimizer when used as a wrapper for adaptive methods that exhibit specific vulnerabilities during the training of deep neural networks. Based on the original formulation by [Zhang et al. 2019], we implemented Lookahead as a general-purpose class in PyTorch and paired it with four base optimizers—SGD with momentum, Adam, Adagrad, and RMSProp—to evaluate whether the slow- and fast-weight interpolation mechanism can mitigate the structural flaws of each method. Experiments were conducted using the ResNet-18 architecture on the CIFAR-10 dataset, analyzing convergence efficiency, the quality of the minima found, and generalization performance.*

Resumo. *O presente trabalho investiga o comportamento do otimizador Lookahead quando utilizado como encapsulador de métodos adaptativos que apresentam vulnerabilidades específicas durante o treinamento de redes neurais profundas. Partindo da formulação original de [Zhang et al. 2019], implementamos o Lookahead como uma classe generalista em PyTorch e o acoplamos a quatro otimizadores base: SGD com momentum, Adam, Adagrad e RMSProp, avaliando se o mecanismo de interpolação de pesos lentos e rápidos é capaz de mitigar falhas estruturais de cada método. Os experimentos são conduzidos sobre a arquitetura ResNet-18 no conjunto de dados CIFAR-10, analisando eficiência de convergência, qualidade dos mínimos encontrados e generalização.*

1. Introdução

O treinamento de redes neurais profundas pode ser formulado como um problema de otimização: dado um conjunto de dados e uma função de perda, busca-se encontrar o conjunto de pesos que minimiza o erro de predição do modelo sobre exemplos não vistos. Nesse contexto, o papel dos otimizadores é central. Formalmente, dado um vetor de parâmetros θ e uma função de perda $L(\theta)$, o objetivo é resolver:

$$\theta^* = argminE[L(f(x;\theta), y] \tag{1}$$

Um otimizador é, portanto, o algoritmo responsável por percorrer iterativamente o espaço de parâmetros em busca de um ponto de mínimo (ou mínimo local suficientemente bom) da função de perda. Em cenários de aprendizado profundo, essa superfície é tipicamente não-convexa, de altíssima dimensionalidade e repleta de pontos de sela, platôs e mínimos locais. A escolha inadequada do otimizador pode levar a convergência prematura, oscilações, ou simplesmente ao não-aprendizado do modelo.

A literatura de otimização para aprendizado profundo evoluiu de forma significativa nas últimas décadas. O Gradiente Descendente Estocástico (SGD) e suas variações com momentum permanecem referências robustas para muitas tarefas. Paralelamente, métodos adaptativos, como Adagrad, RMSProp e Adam, introduziram a noção de taxas de aprendizado por parâmetro, ajustadas automaticamente com base no histórico de gradientes. Cada um desses métodos carrega, contudo, vulnerabilidades específicas: o Adagrad sofre com decaimento excessivo e prematuro da taxa de aprendizado em superfícies não-convexas; o RMSProp pode gerar atualizações divergentes por ausência de correção de viés; o Adam, apesar de robusto, pode falhar em generalizar tão bem quanto o SGD em certas arquiteturas.

Nesse panorama, [Zhang et al. 2019] propuseram o Lookahead, um método ortogonal aos algoritmos existentes. Em vez de substituir o otimizador base, o Lookahead o encapsula, mantendo dois conjuntos de pesos — pesos rápidos (θ) e pesos lentos (φ) — e realizando atualizações interpoladas a cada k passos do loop interno. A intuição central é que os pesos rápidos exploram regiões do espaço de parâmetros com maior liberdade, enquanto os pesos lentos âncoram o processo, evitando oscilações e melhorando a estabilidade do treinamento.

O presente trabalho parte de uma pergunta ainda pouco explorada na literatura: se o Lookahead se comporta bem em combinação com otimizadores clássicos como SGD e Adam, ele seria igualmente eficaz como regulador de otimizadores com falhas estruturais conhecidas? Especificamente, investigamos se o mecanismo de interpolação linear dos pesos lentos é capaz de compensar (i) o decaimento prematuro da taxa de aprendizado do Adagrad e (ii) a instabilidade por ausência de correção de viés do RMSProp, além de comparar com SGD com momentum e Adam como linhas de base.

Para isso, implementamos o Lookahead como um encapsulador generalista em PyTorch, conduzimos experimentos sistemáticos sobre a arquitetura ResNet-18 no conjunto de dados CIFAR-10 e avaliamos três dimensões principais: (i) eficiência de convergência, mensurada pela curva de perda de validação por época; (ii) qualidade dos mínimos encontrados, analisando mínimos planos versus pontiagudos; e (iii) acurácia de validação final.

2. Trabalhos Relacionados

O SGD é o ponto de partida histórico e conceitual da área. [Ruder 2017], apresenta as três variantes principais: gradiente descendente em lote completo (batch GD), gradiente descendente estocástico puro e mini-batch SGD. Este último representa o compromisso dominante na prática, equilibrando precisão do gradiente e eficiência computacional. [Kashyap 2023] aprofunda a análise teórica do SGD, destacando como a trajetória do algoritmo depende fortemente da taxa de aprendizado, do tamanho do lote e da covariância dos gradientes.

A introdução do momentum por [Polyak 1964] marcou uma melhoria significativa sobre o SGD puro. O momentum acumula uma média móvel exponencial das atualizações de gradiente passadas, permitindo ao algoritmo ganhar velocidade em direções consistentes e amortecer oscilações em direções de alta curvatura. Uma variante influente é o Nesterov Accelerated Gradient (NAG), que avalia o gradiente em um ponto à frente da posição atual, proporcionando uma correção mais responsiva e convergência de ordem

O(1/k²) para funções convexas.

Os métodos adaptativos representam a segunda grande família de otimizadores de primeira ordem. O Adagrad, proposto por [Duchi et al. 2011], introduziu taxas de aprendizado individuais por parâmetro, inversamente proporcionais à raiz quadrada da soma acumulada dos quadrados dos gradientes históricos. Isso beneficia parâmetros com gradientes esparsos, mas resulta em decaimento monotônico e potencialmente excessivo da taxa de aprendizado em cenários não-convexos, uma limitação reconhecida que motiva diretamente um dos braços deste estudo.

O RMSProp, proposto por [Tieleman and Hinton 2012], substituiu a soma acumulada do Adagrad por uma média móvel exponencial dos gradientes ao quadrado, contornando o problema de decaimento prematuro. No entanto, ao contrário do Adam, o RMSProp não inclui um mecanismo de correção de viés para a inicialização dos momentos em zero, o que pode gerar estimativas imprecisas nos passos iniciais do treinamento e levar a atualizações instáveis.

O Adam, introduzido por [Kingma and Ba 2015], combina os benefícios do momentum e do RMSProp. Ele mantém médias móveis exponenciais tanto do gradiente (primeiro momento) quanto do quadrado do gradiente (segundo momento), aplicando correções de viés para compensar a inicialização em zero dos vetores de momento. Sua robustez e baixo custo de ajuste de hiperparâmetros tornaram-no o otimizador padrão em muitas aplicações. Contudo, [Wilson et al. 2017] observaram empiricamente que soluções encontradas por métodos adaptativos podem generalizar pior do que aquelas obtidas pelo SGD com momentum afinado.

2.1. O problema dos Mínimos Planos e Pontiagudos

Uma linha de pesquisa crítica para o presente trabalho diz respeito à relação entre a geometria dos mínimos encontrados e a capacidade de generalização dos modelos. [Keskar et al. 2017] demonstraram que métodos de treinamento com lotes grandes tendem a convergir para mínimos pontiagudos, caracterizados por autovalores grandes da Hessiana, os quais generalizam pior do que os mínimos planos encontrados por métodos de lote pequeno. [Hochreiter and Schmidhuber 1997] haviam previamente identificado a importância de mínimos planos para a generalização. [He et al. 2021] sumarizam a controvérsia em torno desse tema, reconhecendo que, embora disputada, a correlação entre planura do mínimo e generalização motiva a criação de algoritmos que ativamente buscam regiões mais planas, como o Entropy-SGD e o SAM (Sharpness-Aware Minimization).

Nesse contexto, a hipótese que guia parte dos experimentos deste trabalho é que o mecanismo de interpolação do Lookahead, ao suavizar as trajetórias dos pesos rápidos via pesos lentos, pode naturalmente guiar o processo de otimização em direção a mínimos mais planos, melhorando a generalização, especialmente quando o otimizador base tem tendência a convergir para soluções instáveis.

3. Fundamentação Teórica

3.1. O problema de Otimização de Redes Neurais

O treinamento de uma rede neural profunda pode ser formalizado como a minimização do risco empírico sobre um conjunto de treinamento. Dado um conjunto de n pares (x_i, y_i), busca-se:

$$\theta^* = argmin(1/n) \sum_i L(f(x_i; \theta), y_i) + R(\theta) \tag{2}$$

onde L é a função de perda (por exemplo, entropia cruzada para classificação), $f(x; \theta)$ é a predição do modelo e $R(\theta)$ é um termo de regularização opcional. Em redes profundas, essa função objetivo é não-convexa, de alta dimensionalidade e altamente sensível à inicialização dos pesos. A regra de atualização geral de um algoritmo de gradiente descendente é:

$$\theta_{t+1} = \theta_t - \eta_t g_t \tag{3}$$

onde η_t é a taxa de aprendizado no passo t e g_t é alguma estimativa (possivelmente modificada) do gradiente. Os diferentes otimizadores se distinguem precisamente pela forma como constroem g_t e como ajustam η_t ao longo do treinamento.

3.2. Otimizadores Base

SGD com Momentum: O momentum acumula uma média móvel exponencial das atualizações de gradiente, definindo o vetor de velocidade $v_t = \beta v_{t-1} + \eta \nabla L(\theta_t)$, com a atualização $\theta_{t+1} = \theta_t - v_t$. O coeficiente $\beta \in [0, 1)$ controla a taxa de decaimento do momentum acumulado. O momentum acelera a convergência ao longo de direções de gradiente consistente e atenua oscilações em direções de alta curvatura, comportando-se analogamente a uma bola descendo uma superfície com inércia.

Adagrad: Adapta individualmente a taxa de aprendizado de cada parâmetro, acumulando os quadrados de todos os gradientes passados em uma matriz diagonal G_t. A atualização é $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \varepsilon}} \odot g_t$, onde ε é uma constante de estabilidade numérica. Parâmetros com gradientes historicamente grandes recebem taxas menores; parâmetros com gradientes pequenos recebem taxas maiores. Em superfícies não-convexas, a acumulação irrestrita de G_t leva ao decaimento monotônico e eventual paralisação da taxa de aprendizado.

RMSProp: Resolve o problema do Adagrad substituindo a soma acumulada por uma média móvel exponencial dos quadrados dos gradientes: $v_t = \rho v_{t-1} + (1 - \rho) g_t^2$, com atualização $\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} g_t$. A janela efetiva de gradientes considerada é controlada por ρ. A ausência de correção de viés, contudo, torna as estimativas imprecisas nas primeiras iterações e pode gerar passos desproporcionalmente grandes.

Adam: Combina o momentum de primeira e segunda ordem com a correção de viés. Mantém médias móveis $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ e $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$, com correções $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ e $\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$. A atualização é $\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \varepsilon}}$. Os valores padrão $\beta_1 = 0{,}9$, $\beta_2 = 0,999$ e $\alpha = 0,001$ funcionam bem em uma ampla gama de problemas sem ajuste fino.

3.3. O Otimizador Lookahead

O Lookahead, proposto por [Zhang et al. 2019] é um algoritmo de otimização ortogonal às abordagens tradicionais. Em vez de modificar o mecanismo interno de

atualização de gradientes, ele opera como um encapsulador em torno de qualquer otimizador padrão A, introduzindo uma segunda escala temporal de atualização por meio de dois conjuntos de pesos:

Pesos rápidos (θ): atualizados a cada iteração pelo otimizador base A, explorando a superfície de perda com a dinâmica habitual do método interno.

Pesos lentos (φ): atualizados a cada k iterações do loop interno por interpolação linear em direção aos pesos rápidos finais.

Para cada ciclo $t = 1, 2, \ldots$, os pesos rápidos são sincronizados com os lentos ($\theta_{t,0} \leftarrow \varphi_{t-1}$) e, a seguir, o otimizador base executa k atualizações internas: $\theta_{t,i} \leftarrow \theta_{t,i-1} + A(L, \theta_{t,i-1}, d)$, onde d é um minibatch amostrado de D. Após os k passos internos, realiza-se a atualização externa dos pesos lentos:

$$\varphi_t = \varphi_{t-1} + \alpha(\theta_{t,k} - \varphi_{t-1}) \tag{4}$$

onde $\alpha \in (0, 1]$ é a taxa de aprendizado dos pesos lentos. Os pesos lentos são então utilizados como novo ponto de partida dos pesos rápidos no próximo ciclo.

3.4. Interpretação da Trajetória dos Pesos Lentos

Uma propriedade fundamental do Lookahead é que a trajetória dos pesos lentos pode ser caracterizada como uma média móvel exponencial (EMA) dos pesos rápidos finais de cada ciclo interno. Após k passos internos, a equação de atualização se expande como:

$$\varphi_{t+1} = \alpha[\theta_{t,k} + (1-\alpha)\theta_{t-1,k} + \ldots + (1-\alpha)^{t-1}\theta_{0,k}] + (1-\alpha)^t\varphi_0 \tag{5}$$

Essa formulação revela que os pesos lentos atribuem peso exponencialmente decrescente às propostas passadas dos pesos rápidos, mantendo coerência com a teoria das médias de [Polyak 1964] e diferenciando-se do SWA, que computa médias aritméticas.

3.5. Complexidade Computacional

Uma das propriedades mais atraentes do Lookahead é seu custo computacional negligenciável. O número de operações é $O\frac{(k+1)}{k}$ vezes o do otimizador interno — ou seja, apenas $\frac{1}{k}$ de operações adicionais por iteração, amortizadas ao longo dos k passos internos. Em termos de memória, o Lookahead mantém uma cópia adicional dos parâmetros treináveis do modelo (os pesos lentos), o que representa um acréscimo fixo de memória independente de k.

3.6. Tratamento do Estado Interno do Otimizador

Uma questão de implementação relevante é o que fazer com o estado interno do otimizador base (por exemplo, o vetor de momentum do SGD ou os momentos do Adam) após cada atualização dos pesos lentos. [Zhang et al. 2019] exploram três estratégias: (i) manter o estado, sem qualquer modificação; (ii) interpolar o estado, aplicando a mesma interpolação linear dos pesos; e (iii) resetar o estado, reiniciando o acumulador do otimizador base a cada ciclo de k passos. Nos experimentos sobre o CIFAR, este trabalho também realizou uma comparação e análise entre as estratégias.

4. Metodologia

4.1. Objetivo e Hipóteses de Pesquisa

O objetivo central deste estudo é investigar empiricamente se o Lookahead, quando utilizado como encapsulador generalista, é capaz de atuar como um regulador eficaz para otimizadores com vulnerabilidades estruturais conhecidas. Derivamos três hipóteses principais:

H1 (Convergência): O Lookahead melhora a eficiência de convergência de todos os otimizadores base testados, reduzindo o número de épocas necessárias para atingir determinado nível de perda de validação.

H2 (Estabilidade): O Lookahead reduz a variância entre diferentes execuções para todos os otimizadores base, evidenciando o efeito de redução de variância.

H3 (Mínimos Planos): O Lookahead tende a guiar o treinamento para mínimos mais planos, mensuráveis indiretamente pela acurácia de teste e pela diferença entre acurácia de treino e de validação.

4.2. Implementação do Lookahead como Encapsulador

Implementamos o Lookahead como uma classe Python que herda de torch.optim.Optimizer, seguindo a formulação do Algoritmo 1 de [Zhang et al. 2019]. A classe recebe como parâmetros: (i) o otimizador base instanciado; (ii) k, o número de passos internos por ciclo; (iii) α, a taxa de interpolação dos pesos lentos; e (iv) uma estratégia de gerenciamento do momentum do otimizador base. A implementação mantém, para cada parâmetro do modelo, um estado próprio com os pesos lentos (`cached_params`), com a mesma estrutura de param_groups do otimizador encapsulado. O método step() executa o passo do otimizador base a cada chamada e checa se k passos internos foram completados; quando positivo, executa a interpolação dos pesos lentos sobre todos os grupos de parâmetros. A escolha de $k = 5$ e $\alpha = 0,5$ como valores padrão segue a recomendação do artigo original.

4.3. Arquitetura e Conjunto de Dados

Todos os experimentos utilizam a arquitetura ResNet-18, conforme proposta por [He et al. 2016], partindo de pesos pré-treinados no ImageNet (`ResNet18_Weights.DEFAULT`), com a camada final (fc) substituída por uma camada linear com 10 saídas, correspondentes às classes do CIFAR-10. Trata-se, portanto, de um regime de *fine-tuning* de uma rede pré-treinada, e não de um treinamento do zero como no trabalho original de [Zhang et al. 2019]. O conjunto de dados utilizado é o CIFAR-10, composto por 60.000 imagens coloridas de 32×32 pixels distribuídas em 10 classes, com 50.000 amostras para treinamento e 10.000 para teste. O pré-processamento aplicado consistiu apenas na normalização dos valores de pixel para o intervalo $[0, 1]$ (divisão por 255) e no redimensionamento das imagens de 32×32 para 224×224 pixels via interpolação bilinear, necessário para compatibilizar as imagens do CIFAR-10 com a entrada esperada pela ResNet-18 pré-treinada no ImageNet. Não foram aplicadas técnicas de *data augmentation* (como preenchimento seguido de recorte aleatório ou espelhamento horizontal), nem normalização por média e desvio padrão por canal, diferindo do protocolo de [Zagoruyko and Komodakis 2017].

4.4. Configuração dos Experimentos

Realizamos experimentos em dois grupos de condições: (i) otimizadores puros, como linha de base, e (ii) otimizadores envelopados pelo Lookahead. Os quatro otimizadores base avaliados são: SGD com momentum, Adam, Adagrad e RMSProp. Os hiperparâmetros efetivamente utilizados foram: para o SGD, taxa de aprendizado $0,01$, momentum $0,9$ e *weight decay* de 5×10^{-4}; para o Adam, taxa de aprendizado $0,001$, sem *weight decay*; para o Adagrad, taxa de aprendizado $0,01$; e para o RMSProp, taxa de aprendizado $0,001$ e $\alpha = 0,99$. O Lookahead é aplicado com $k = 5$ e $\alpha = 0,5$ sobre cada um dos quatro otimizadores base, resultando em 8 configurações no total (4 puras + 4 envelopadas).

Os modelos foram treinados por 10 épocas, com mini-batches de 64 imagens, sem decaimento programado da taxa de aprendizado (não há uso de *scheduler* como StepLR ou MultiStepLR). Diferentemente do protocolo original de [Zhang et al. 2019], que utiliza 3 seeds distintas por configuração selecionadas sem critério específico, cada configuração foi executada com três seeds aleatórias distintas (42, 123 e 2024), fixadas tanto na inicialização dos pesos da camada final quanto na ordem de amostragem do DataLoader, permitindo estimar a média e o desvio padrão da acurácia final de cada configuração. As execuções foram conduzidas em ambiente Kaggle Notebooks, utilizando duas GPUs NVIDIA Tesla T4.

4.5. Avaliação do Comportamento do Momentum

Um experimento adicional avalia o impacto da estratégia de gerenciamento do momentum do otimizador base no momento da sincronização dos pesos lentos. Especificamente, comparamos três estratégias: manter o momentum, interpolar o momentum proporcionalmente à interpolação dos pesos, e resetar o momentum a zero após cada atualização dos pesos lentos, para o SGD com momentum envelopado pelo Lookahead. Este é o único otimizador, entre os quatro avaliados, cujo estado interno mantido pelo PyTorch (`momentum_buffer`) corresponde a um momentum clássico sobre o qual essas três estratégias têm efeito real. Para os otimizadores adaptativos (Adam, Adagrad e RMSProp), o estado interno é mantido em variáveis com outra semântica (respectivamente `exp_avg`/`exp_avg_sq`, `sum` e `square_avg`), que não correspondem a um momentum no sentido em que as estratégias de reset e interpolação foram definidas. Por essa razão, esse experimento foi conduzido apenas para o SGD, também com as três seeds (42, 123 e 2024). Para os demais otimizadores, reportamos exclusivamente os resultados mantendo o comportamento padrão do momentum, sem as variantes de reset ou interpolação.

5. Resultados

A Tabela 1 apresenta a acurácia final média (%) e o desvio padrão obtidos em 3 execuções (seeds 42, 123 e 2024) para cada otimizador base, comparando a versão pura (Baseline) e a versão envelopada pelo Lookahead (`pullback_momentum='none'`), além das duas variantes de gerenciamento de momentum testadas exclusivamente para o SGD.

A Figura 1 apresenta a convergência da perda para cada uma das três seeds, e a Figura 2 apresenta a acurácia final correspondente, ambas comparando Baseline e Lookahead para os quatro otimizadores avaliados.

Tabela 1. Acurácia final (%), média ± desvio padrão sobre 3 seeds (42, 123, 2024)

Otimizador	Configuração	Acurácia (%)
SGD	Baseline	$92,61 \pm 0,73$
	Lookahead (none)	$95,66 \pm 0,15$
	Lookahead (reset)	$95,05 \pm 0,04$
	Lookahead (pullback)	$95,31 \pm 0,42$
Adam	Baseline	$89,95 \pm 0,86$
	Lookahead (none)	$90,39 \pm 2,15$
Adagrad	Baseline	$88,90 \pm 3,32$
	Lookahead (none)	$86,13 \pm 13,52$
RMSProp	Baseline	$83,54 \pm 4,13$
	Lookahead (none)	$76,24 \pm 6,67$

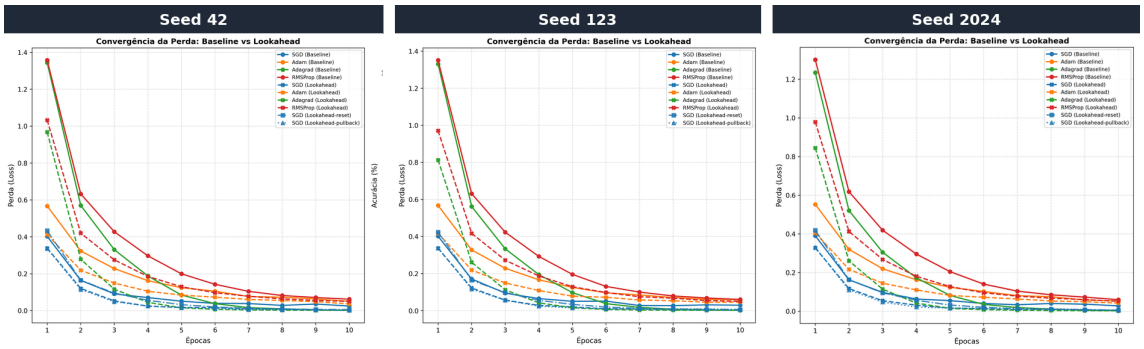


Figura 1. Convergência da perda, Baseline vs Lookahead, para as seeds 42, 123 e 2024

6. Discussão

6.1. SGD

Para o SGD com momentum, o Lookahead produziu ganho consistente de acurácia nas 3 seeds testadas ($+2,61$, $+3,87$ e $+2,67$ pontos percentuais, respectivamente, Figura 1), com desvio padrão baixo tanto no Baseline ($\pm 0,73$) quanto no Lookahead ($\pm 0,15$). Esse comportamento é coerente com o fato de o SGD ser, entre os quatro otimizadores avaliados, o único cujo estado interno (`momentum_buffer`) corresponde a um momentum clássico sobre o qual o Lookahead consegue de fato atuar de forma completa, tanto na interpolação dos pesos quanto na gestão do momentum. As três estratégias de gerenciamento do momentum (`none`, `reset` e `pullback`), visíveis na Figura 2, apresentaram resultados muito próximos entre si ($95,66\%$, $95,05\%$ e $95,31\%$, respectivamente), sugerindo que, para o SGD, o ganho do Lookahead é robusto à escolha dessa estratégia. Esse resultado corrobora a conclusão de [Zhang et al. 2019] de que o método é pouco sensível a esse hiperparâmetro.

6.2. Adam

O Adam apresentou o comportamento mais estável entre os otimizadores adaptativos, com ganho médio pequeno ($+0,44$ ponto percentual) e desvio padrão de $\pm 2,15$ no Lookahead, maior que o do Baseline ($\pm 0,86$), mas sem inversões de sinal expressivas entre seeds. Apenas a seed 123 apresentou queda de $-1,79$ ponto, visível na Figura 2.

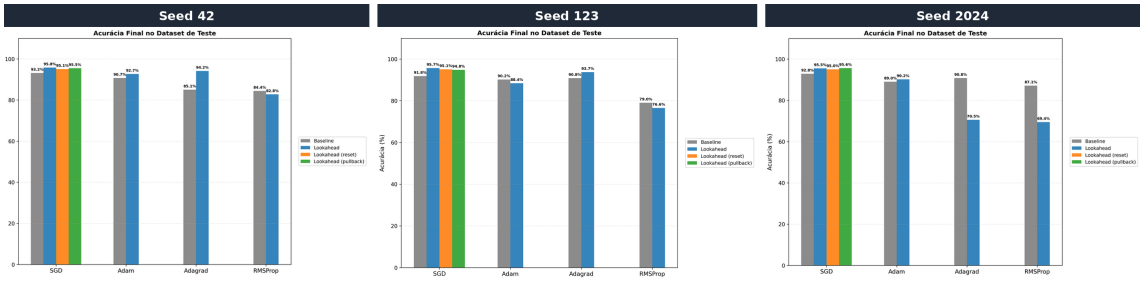


Figura 2. Acurácia final, Baseline vs Lookahead, para as seeds 42, 123 e 2024

6.3. Adagrad e RMSProp

O achado mais relevante deste trabalho está nos otimizadores Adagrad e RMSProp. Nas seeds 42 e 123 (Figura 2), o Lookahead melhorou a acurácia do Adagrad de forma expressiva ($+9,09$ e $+2,90$ pontos, respectivamente). Na seed 2024, entretanto, a mesma configuração apresentou uma queda abrupta de $-20,28$ pontos percentuais ($90,80\%$ para $70,52\%$), apesar de a perda de treinamento final ter sido menor que a do Baseline. O mesmo padrão ocorreu com o RMSProp na seed 2024 ($-17,72$ pontos, de $87,14\%$ para $69,42\%$), também com perda de treinamento reduzida em relação ao Baseline.

Esse padrão, perda de treinamento baixa acompanhada de acurácia de teste muito inferior, é consistente com um cenário de *overfitting* ou de perda de generalização, sem que a curva de perda por época (monotonamente decrescente, sem picos, Figura 1) indique instabilidade numérica durante o treinamento. Uma explicação plausível é que, diferentemente do SGD, o Lookahead não gerencia os estados internos adaptativos do Adagrad (`sum`) e do RMSProp (`square_avg`) durante a interpolação periódica dos pesos. Apenas os pesos do modelo são puxados de volta em direção à média, enquanto as estatísticas acumuladas do otimizador base continuam evoluindo de forma independente. Essa dessincronia entre o estado dos pesos e o estado adaptativo do otimizador pode, dependendo da inicialização e da ordem de amostragem dos dados, levar o modelo a mínimos que minimizam bem a perda de treinamento mas generalizam mal. Esse comportamento só se torna visível ao comparar múltiplas seeds, e passaria despercebido caso o trabalho se limitasse a uma única execução.

7. Conclusão

Este trabalho implementou o Lookahead como um encapsulador de otimizadores em PyTorch e avaliou seu efeito sobre quatro otimizadores base (SGD, Adam, Adagrad e RMSProp) no treinamento de uma ResNet-18 pré-treinada sobre o CIFAR-10, com 3 seeds distintas por configuração.

Os resultados indicam que os benefícios do Lookahead relatados por [Zhang et al. 2019] não se estendem de forma homogênea a todos os otimizadores base. Para o SGD com momentum, observou-se ganho de acurácia consistente e de baixa variância entre execuções, robusto inclusive à estratégia de gerenciamento do momentum (`none`, `reset` ou `pullback`). Para o Adam, o ganho foi pequeno e a variância entre seeds moderada. Já para o Adagrad e o RMSProp, o comportamento foi marcado por alta variância: em duas das três seeds testadas, o Lookahead melhorou ou manteve a

acurácia; na terceira (seed 2024), ambos os otimizadores apresentaram quedas expressivas de acurácia de teste (superiores a 17 pontos percentuais) mesmo com perda de treinamento reduzida em relação ao Baseline, sugerindo perda de generalização.

Esse resultado reforça a importância metodológica de executar múltiplas seeds antes de generalizar conclusões sobre o efeito de uma técnica de otimização. Uma análise baseada em uma única execução poderia ter levado a conclusões opostas, dependendo apenas de qual seed tivesse sido escolhida. Como hipótese explicativa, aponta-se a ausência, na implementação avaliada, de um mecanismo de sincronização entre os pesos lentos do Lookahead e os estados internos adaptativos do Adagrad e do RMSProp, presente apenas para o SGD, via o `momentum_buffer`.

Como trabalhos futuros, sugerem-se a investigação de estratégias de sincronização de estado adaptativo do Adagrad e do RMSProp análogas às testadas para o momentum do SGD e a repetição dos experimentos com o protocolo original de [Zhang et al. 2019] (pesos inicializados aleatoriamente, 200 épocas, *data augmentation* completo e decaimento programado da taxa de aprendizado), de modo a isolar o efeito do regime de *fine-tuning* adotado neste trabalho como possível fator de instabilidade adicional.

Referências

Duchi, J., Hazan, E., and Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159.

He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778.

He, X., Xue, F., Ren, X., and You, Y. (2021). Large-scale deep learning optimizations: A comprehensive survey.

Hochreiter, S. and Schmidhuber, J. (1997). Flat minima. *Neural Computation*, 9(1):1–42.

Kashyap, R. V. (2023). A survey of deep learning optimizers – first and second order methods. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 7(1).

Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*.

Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17.

Ruder, S. (2017). An overview of gradient descent optimization algorithms.

Tieleman, T. and Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude. *COURSERA: Neural Networks for Machine Learning*, 4(2):26–31.

Wilson, A. C., Roelofs, R., Stern, M., Srebro, N., and Recht, B. (2017). The marginal value of adaptive gradient methods in machine learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30.

Zagoruyko, S. and Komodakis, N. (2017). Wide residual networks.

Zhang, M. R., Lucas, J., Hinton, G., and Ba, J. (2019). Lookahead optimizer: k steps forward, 1 step back. In *Advances in Neural Information Processing Systems*, volume 32.